

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE
LAUSANNE

Improving Classification Performance of Convolutional Neural Networks by Changing the Convolutional Kernel Shape

Student:

Firas BEN OTHMAN

Supervisor:

Dr. Bastien PASDELOUP



ÉCOLE POLYTECHNIQUE
FÉDÉRALE DE LAUSANNE

January 11, 2019

1 Introduction

Convolutional Neural Networks are very widely used nowadays for their efficiency in classification tasks. However, the kernel/filter of the network is usually arbitrarily shaped as an $N \times M$ rectangle and translated over the image to detect patterns in the data.

But, what effect would a kernel of an alternative shape have on the prediction accuracy and the quality of the results of the Network?

Some studies [Lassance et al., 2018][Sun et al., 2016] have experimented with replacing the kernel shape by an alternative diamond shape or slightly deformed squares.

The results of these studies were mainly slightly better accuracies. This promises better accuracies overall by exploiting further more complex kernel shapes.

In this project, we explore different and more complex options of kernel geometrical shapes other than rectangular or square and examine how it affects prediction accuracy compared to the ordinary convolution with a square kernel.

This has been done by implementing the convolution with non-square kernel shapes using the kernel location file which will be explained below.

Then, a simple network was built using the convolution with non-square shaped kernels and some dense linear layers. Then, it was used to learn and predict the labels of images of a dataset of simple geometrical shapes as well as the MNIST dataset later on.

Finally, a discussion of the performance of the predictions was done for each kernel shape used for the experiments comparing it to the square shaped kernel performance.

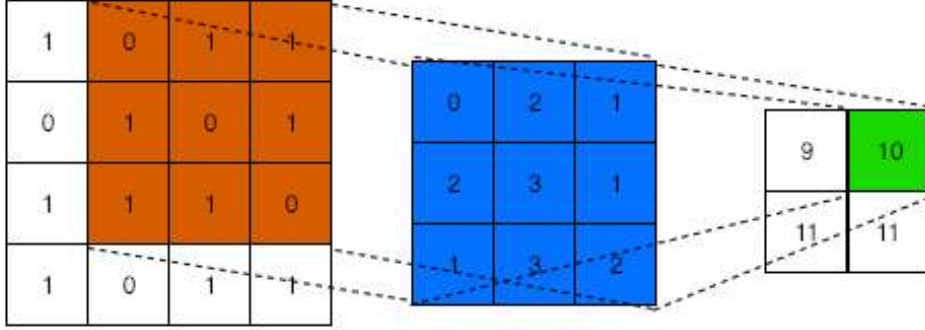
2 Introducing kernels with non-square shapes in convolutions

In the classical case, Convolutional Neural Networks have a $N \times M$ kernel which is translated around the convolutional layer to produce the next layer. A graphical example of this can be seen in Figure 1 below.

What we want to do is create a convolution that uses a non-square/non-

rectangle shaped kernel. The convolution in this case will be done by assigning a center to the kernel and placing it around the image and eventually producing the result of the convolution as the next layer.

Figure 1: Classical Convolutional Neural Networks



In order to use non-rectangular shaped kernel, a kernel location file was generated to indicate the translation of the kernel over the image in a convolutional layer. Then, to simulate a Convolutional Neural Network a Ternary layer architecture that makes use of the generated kernel location file.

2.1 Kernel location file

2.1.1 Kernel Definition

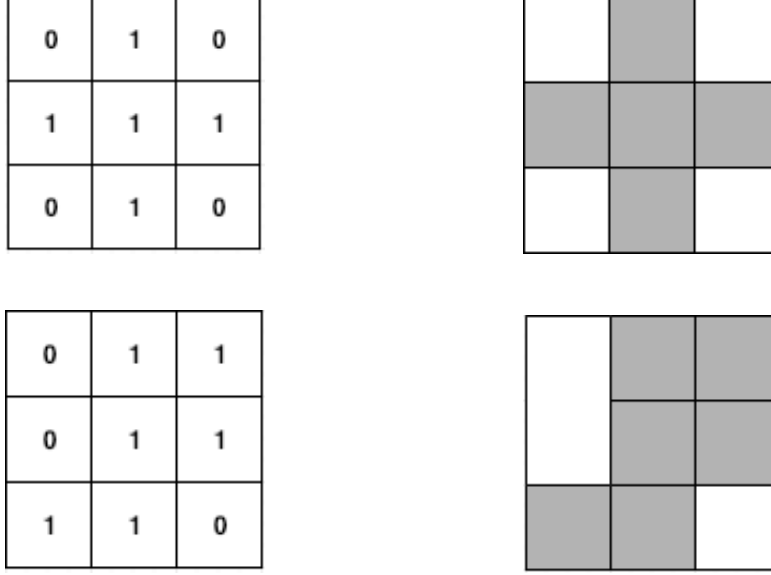
The kernel shape was introduced as a tensor containing 1s to indicate the presence of pixels and 0s to indicate the absences.

This is visually explained in Figure 2 below. To the left, you can find the tensor passed and to the right the corresponding kernel shape in grey.

2.1.2 Weights Assignment

The kernel pixels have been assigned some weights. The center of the kernel is assigned w_1 while all the other pixels have been assigned weights ranging from w_2 to w_n , n being the number of pixels of the kernel. Refer to Figure 3.

Figure 2: Tensors and their corresponding kernel shapes



2.1.3 Kernel location file generation

The convolutional layer associated with a given kernel shape is represented as a list of pixels enumerated from 0 to N-1. The pixels are then covered by the kernel weights and translated over all the image. The center of the kernel is placed iteratively on the pixels of the convolutional layer. As a result, the other weights are usually placed on other pixels of the image. The kernel location file keeps track of all the pixels covered by specific weights at each translation of the kernel in the convolution process.

This is done by keeping a list of dictionaries each containing the weight index as a key and the pixel index as a corresponding value.

This file will then be used to compute the convolution of the layer by the kernel.

For instance, using the kernel introduced in Figure 3, at the very beginning pixel 0 is covered by w_1 , pixel 1 is covered by w_5 and pixel W is covered by w_3 . W being the width of an image.

After translating by one pixel, the pixel 1 would be covered by w_1 , pixel 2 by w_5 , pixel $1 + W$ by w_3 and pixel 0 by w_4 . So far, we'll have a kernel location file that looks like this : $\{1 : 0, 3 : W, 5 : 1\}, \{1 : 1, 3 : W + 1, 4 : 0,$

Figure 3: weights assignment

	w_2	
w_4	w_1	w_5
	w_3	

5 : 2}].

The keys of each dictionary are the weights number and values correspond to the pixel of the image covered by the weight.

At the end of the kernel location file generation, the resulting file should contain a list of N dictionaries.

This is better explained visually in Figures 4 and 5.

Figure 5: kernel location file at 22nd and last translations

0	1	2	3	4	5	6	7	8	9
10	w_2	12	13	14	15	16	17	18	19
w_4	w_1	w_5	23	24	25	26	27	28	29
30	w_3	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69
70	71	72	73	74	75	76	77	78	79
80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99

At the 22nd translation, w_1 covers pixel 21, w_5 pixel 22, w_3 pixel 31, w_2 pixel 11 and w_4 pixel 20. The following dictionary is added to the kernel location file.

{1 : 21 , 2 : 11 , 3 : 31 , 4 : 20 , 5 : 22}

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69
70	71	72	73	74	75	76	77	78	79
80	81	82	83	84	85	86	87	88	w_2
90	91	92	93	94	95	96	97	w_4	w_1

At the last translation, w_1 covers pixel 99, w_2 pixel 89, w_4 pixel 98. The following dictionary is added to the kernel location file.

{1 : 99 , 2 : 89 , 4 : 98}

Figure 4: kernel location file at the first and second translations

w_1	w_5	2	5	4	6	8	7	8	9
w_3	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69
70	71	72	73	74	75	76	77	78	79
80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99

At the beginning, w_1 covers pixel 0, w_5 pixel 1 and w_3 pixel 10.

The kernel location file currently contains
 $[\{ 1 : 0 , 3 : 10 , 5 : 1 \}]$

w_4	w_1	w_5	3	4	5	6	7	8	9
10	w_3	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69
70	71	72	73	74	75	76	77	78	79
80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99

Then, w_1 covers pixel 1, w_5 pixel 2, w_4 pixel 0 and w_3 pixel 10.

The kernel location file currently contains
 $[\{ 1 : 0 , 3 : 10 , 5 : 1 \} ,$
 $\{ 1 : 1 , 3 : 11 , 4 : 0 , 5 : 2 \}]$

2.2 Convolutional Layers

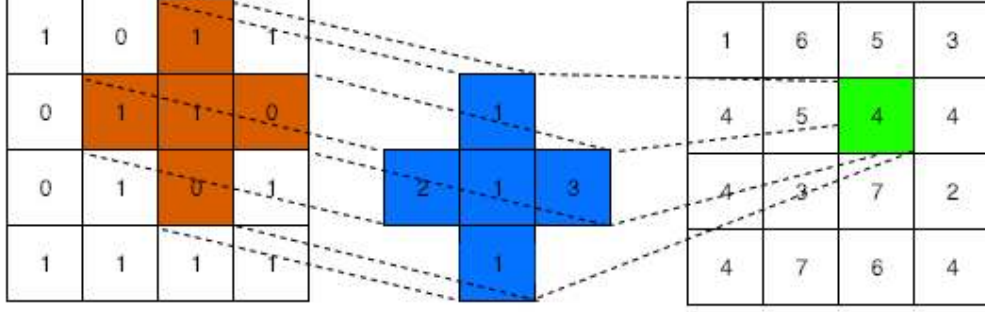
Now that the kernel location file is generated, it is easy to determine which weights of the kernel covers which pixels.

The convolution between two layers is done by moving the kernel over all of the image and computing the resulting pixel at the position of the assigned center of the kernel (called w_1 in previous examples.) by calculating the weighted sum of the weights and their pixels covered at a given translation. This is done easily by referring to the kernel location file which contains all necessary information about weights and corresponding pixels at each translation.

In short, if we have a pixel P that has a value p_1 surrounded by p_2 , p_3 and p_4 which are covered respectively by kernel weights w_1 , w_2 , w_3 and w_4 , the pixel P in the next layer should have a value of :

$$p_1 w_1 + p_2 w_2 + p_3 w_3 + p_4 w_4$$

Figure 6: Example of a forward transition between two consecutive convolutional layers.



In this example the center w_1 has the value 1, when placed on pixel 6, as highlighted in red, it produces the following convolution $1 * 1 + 1 * 1 + 0 * 1 + 1 * 2 + 0 * 3 = 4$. The result of this convolution is at pixel 6 in green at the output layer.

3 Experiments

3.1 Neural Network Architecture

The model chosen for these experiments is a Neural network consisting of a convolutional layer where our kernels will be introduced and four fully connected linear layers.

This network is chosen for its simplicity which enables us to perform multiple experiments with a relatively small execution time. No dropouts were introduced

3.2 Dataset

The dataset used for this project's experiments is a set of images of four shapes found on kaggle which is in summary a set of 200x200 pixels images of triangles, rectangles, circles and stars with different rotations and positions in black and white.

Samples of the dataset can be found in the Figures below.

It contains 3721 images of circles, 3767 images of squares, 3766 images of stars and 3721 images of triangles. This gives us a dataset of 14975 samples.

For each label, 50% of the samples were selected for training and the remaining 50% for testing.

This dataset was chosen for the simplicity of the shapes to be detected. This allows us to use some logic in the choice of the kernel with the odd shapes. For instance, a kernel with a sharp edge should be very useful to detect squares and triangles.

Figure 7: Samples of shapes in Dataset



Sample of a rectangle image.



Sample of a circle image.



Sample of a triangle image.



Sample of a star image

3.3 Introducing our own kernels

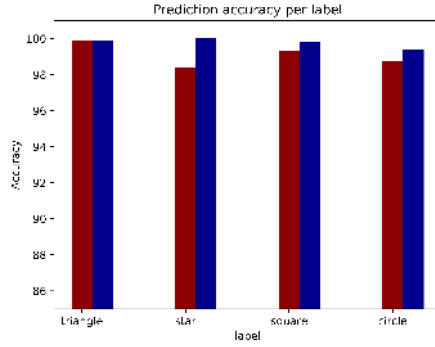
The kernel shapes were chosen intuitively to predict the labels of the dataset samples described above.

Some shapes, such as, corner kernel, cross kernel and line kernel were chosen to detect sharp edges in triangles, squares, and star...

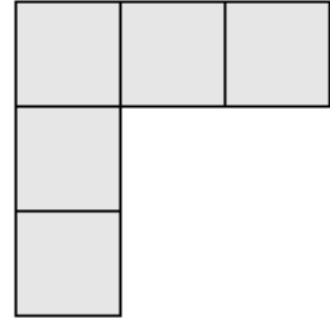
On the other hand, the kernels shaped like triangles, star and circle were chosen as well to see how it affects the prediction accuracy of the particular labels in the dataset.

Next, we show figures of the accuracies of the predictions of each label in the testing set for each chosen kernel shape compared to the accuracies of the predictions with the classical square shapes as bar plots. The performance of the classical neural networks will be shown in as a bar in dark blue while the performance of our neural network in dark red. In addition, the picture of the introduced kernel will be shown next to each bar chart.

Figure 8: Corner Kernel accuracies



Prediction accuracies with corner kernel (in dark red)
vs square kernel (in dark blue).

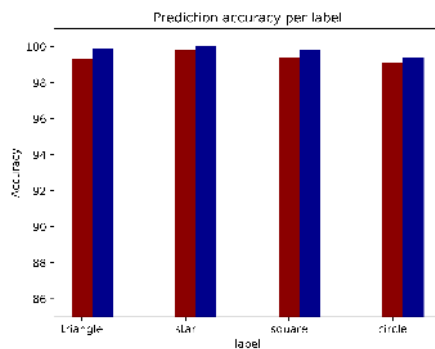


Corner shaped kernel.

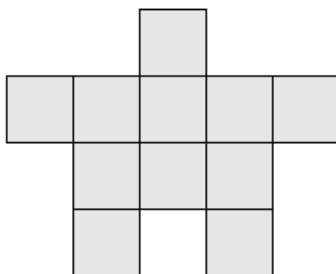
The corner kernel seems to perform as well as the square kernel for correctly predicting triangle images. But , performs slightly worse for predicting the other labels.

This is surprising since this kernel was chosen specifically to detect sharp edged shapes, such as, squares and stars.

Figure 9: Star Kernel accuracies



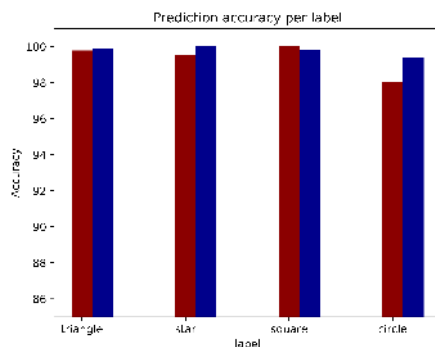
Prediction accuracies with star kernel (in dark red)
vs square kernel (in dark blue).



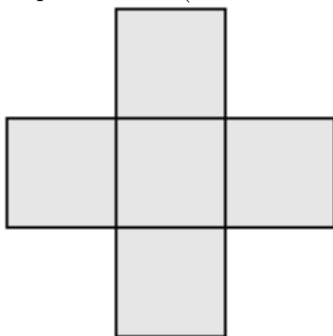
star shaped kernel.

The star shaped kernel performs very similarly to the square kernel.
This is no surprise as the star shaped kernel is very similar to the square
kernel in terms of geometrical form and number of weights.

Figure 10: Cross Kernel accuracies



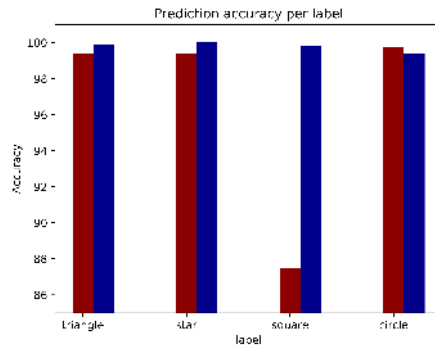
Prediction accuracies with cross kernel (in dark red)
vs square kernel (in dark blue).



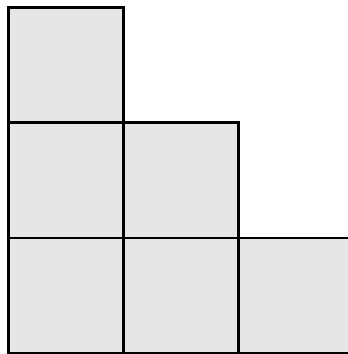
cross shaped kernel.

The cross shaped kernel perform badly for predicting circles, but performs as well as the normal kernel in predicting sharp edges geometrical forms.

Figure 11: Triangle Kernel accuracies



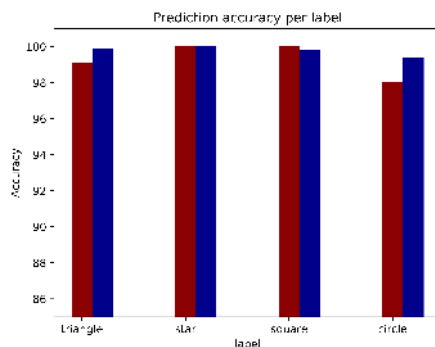
Prediction accuracies with triangle kernel (in dark red) vs square kernel (in dark blue).



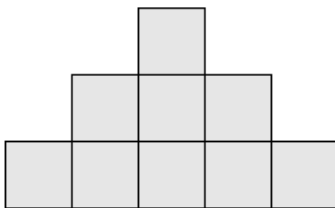
Triangle shaped kernel.

The triangle kernel performs very badly in predicting the square images. It is surprising how badly it performs in predicting square images compared to circle images.

Figure 12: second triangle Kernel accuracies



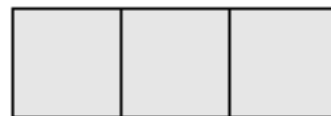
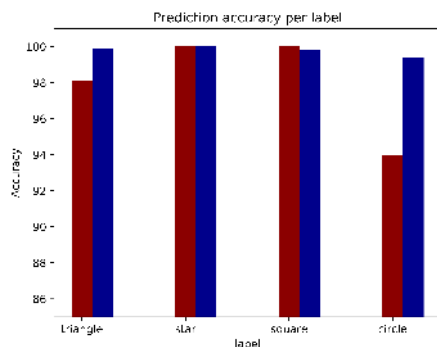
Prediction accuracies with second triangle kernel (in dark red) vs square kernel (in dark blue).



triangle shaped kernel.

The second triangle performs much better and very closely to the square kernel and much better than the first triangular kernel. This may be due to the number of weights of this kernel which is the same as the square kernel's.

Figure 13: Line Kernel accuracies

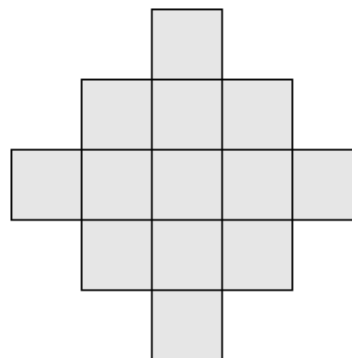
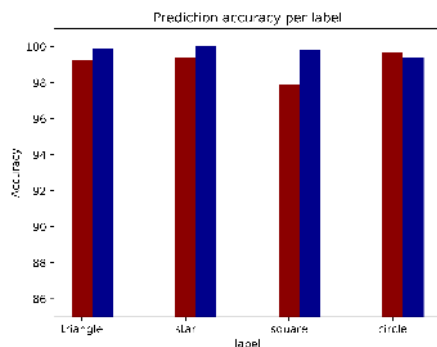


line shaped kernel.

Prediction accuracies with line kernel (in dark red)
vs square kernel (in dark blue).

The line kernel perform really well in predicting the sharp-edged geometrical shapes images but comparatively bad in predicting the circle images. This is no surprise, as this kernel was chosen to see how well it predicts these labels.

Figure 14: Circle Kernel accuracies

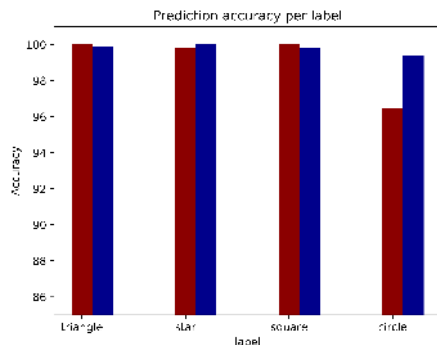


circle shaped kernel.

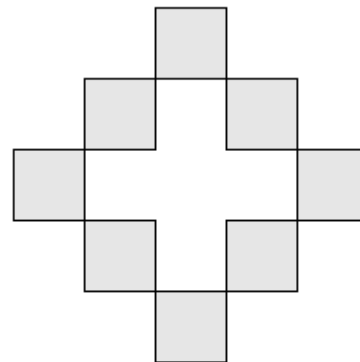
Prediction accuracies with circle kernel (in dark red)
vs square kernel (in dark blue).

The circle kernel performs the best in predicting circle images as expected. In addition, it also predicts really well all of the other labels.

Figure 15: Ring Kernel accuracies



Prediction accuracies with ring kernel (in dark red)
vs square kernel (in dark blue).



ring shaped kernel.

The ring kernel performs very closely to the line kernel. It does not predict well the circles but predicts all the other labels with high accuracy.

3.4 Experiments on MNIST

Due to the simplicity of the images of the shapes dataset, the overall prediction accuracy has always performed an accuracy of above 99%. This was the case even when the prediction for circles was relatively low.

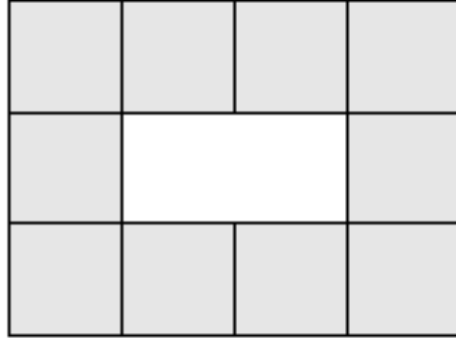
It should be considered that the difference in accuracies may have been due to the randomness of the weight initializations.

That is why, we decided to perform experiments on the MNIST dataset as well. This will allow us to see the impact of the choice of the kernel shape on the accuracy of predicting the digits of the MNIST dataset.

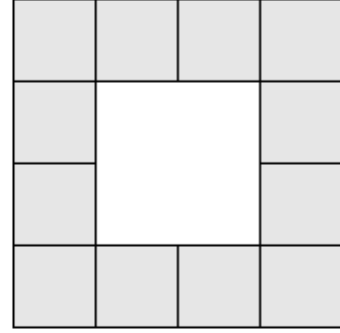
The experiment was performed with the same neural network architecture and the same shapes with the addition of two new shapes which were found to give the best accuracy.

These two shapes are hollow rectangle and hollow square which are shown in the figure blow.

Figure 16: Newly introduced Kernels



Hollow rectangle kernel



Hollow square kernel.

Performance of the kernel on the MNIST dataset

Kernel shape	Accuracy
Square	79.43%
Cross	80.1%
Circle	79.83%
Triangle	78.93%
Second triangle	80.05%
Corner	81.43%
Line	81.42%
Hollow rectangle	82%
Hollow square	81.1%

We can see that most of the kernels perform better than the square kernel. All but the triangle surpass the classical convolutional in accuracy. In the cases of the corner, line and hollow rectangle kernels, the accuracy is superior by at least 2%. That is a significant difference given that the MNIST dataset has 10 distinct labels.

This ensures and show us more clearly that the kernel shapes can impact deeply the performance of the convolutional neural networks on the right dataset.

4 Conclusions

In this project, a convolutional neural network with odd shapes was implemented and tested on two different datasets. It clearly showed that the kernel shape has an impact on the performance of the learning of the datasets for better or for worse.

It is therefore worth investigating which kernels perform best on which datasets. This can possibly be done using dictionary learning to learn the optimal kernel shapes to be used in the learning process.

One more interesting use case could be exploiting these kernels to produce better quality saliency maps on different images than the standard CNN.

References

- [Lassance et al., 2018] Lassance, C. E. R. K., Vialatte, J.-C., and Gripon, V. (2018). Matching convolutional neural networks without priors about data. *arXiv preprint arXiv:1802.09802*.
- [Sun et al., 2016] Sun, Z., Ozay, M., and Okatani, T. (2016). Design of kernels in convolutional neural networks for image classification. In *European Conference on Computer Vision*, pages 51–66. Springer.